# SCA Security Overview

The security measures in the backend and frontend, how the frontend handles data, and the security risks that remain.

Frontend: **sca-fe** (React 19 + Vite, on Vercel) · Backend: **sca-be** (Node 20 + Express 5, on Render) · Data: **Firebase Firestore** + **Cloudflare R2** · Reviewed: 30 October 2026


1. Security at a glance
2. How data flows through the system
3. Backend security measures
4. Frontend: how it handles data
5. Frontend security measures
6. Infrastructure and secrets
7. Possible security risks
8. Recommended next steps


## 1. Security at a glance

| AES-256 | 16 | 14 | 3 |
|---|---|---|---|
| All user records and files encrypted at rest | Security risks reviewed in section 7 | Risks fixed in code | Steps left for the owner (section 8) |

**Still most urgent (owner action):** the Firestore security rules in `sca-be/firestore.rules` (block all direct access) were **never deployed**: the deploy failed because the Google account had no permission on the project. Check the rules in the Firebase console today (see risk R1).

## 2. How data flows through the system

| | |
|---|---|
| **Browser (React)** | Shows forms and lists. Holds the login token and the decrypted data of the signed-in user only while the page is open. Never talks to the database directly. |
| **HTTPS** | Every request goes to `https://sca-web-be.onrender.com/api/v1` over TLS (Vercel and Render both force HTTPS). |
| **API (Express)** | Checks rate limits, the login token, the role and the input; then the service **encrypts before saving** and **decrypts after reading**. |
| **Firestore** | Stores records. User-owned records are `{ reference number, data: <encrypted>, createdAt, updatedAt }`. Only the backend (Admin SDK) can reach it. |
| **Cloudflare R2** | Stores files, photos and media, encrypted on the server before upload. Storage keys are random and never sent to the browser. |

## 3. Backend security measures

| Area | What is in place |
|---|---|
| **Transport & headers** | HTTPS (Render). `helmet` sets security headers. CORS only allows the frontend origin (`CLIENT_URL`). JSON bodies limited to 100 KB. |

| | |
|---|---|
| **Authentication** | Email + password: passwords hashed with **bcrypt (12 rounds)**; new passwords need 8–72 characters with letters and numbers and can't be a common password. Email must be confirmed. **Two-step sign-in:** after the password, a 6-digit code is emailed (10 minutes, 5 tries). Google: the browser signs in with Firebase Auth and the backend **verifies the Firebase ID token** before issuing its own token. Sessions are **JWT** signed with `JWT_SECRET`, valid for 1 day, with a session id and version checked on every request: **sign out** ends that session immediately, **sign out of all devices** and password resets end every session. |
| **Email links** | Confirmation, password reset and developer invitations use one-time random tokens. Only a **SHA-256 hash** is stored; links expire (reset 1 hour, confirmation 24 hours, invitation 72 hours) and are deleted after use. Forgot-password, resend and **sign-up** never reveal whether an email or mobile number exists (the owner is told by email instead). |
| **Authorization (RBAC)** | Roles `sca-1`, `sca-2`, `sca-3`, `dev` (an account can have several). Developer endpoints use `requireRole("dev")`, which reads the role from the database on every request. An existing account only becomes a developer when its owner **accepts the emailed invitation**; every role change is logged. |
| **Ownership** | User data lives at `users/{userId}/...` and services only build paths from the signed-in user's id, so one user can never read another user's records (wrong id → `404`). |
| **Input validation** | Every body, query and URL parameter is validated with **Zod** (types, lengths, formats, required fields); unknown fields are dropped. Ids must match the Firestore id format. |
| **Encryption at rest** | **AES-256-GCM** with a random IV per value (`ENCRYPTION_KEY`). Fully encrypted: user name, email, job title, school, phone, address; students, cases, appointments, files info, categories, To Do board; support reports and system updates; invitation emails; all files/photos/media in R2. Keys can be **rotated** (`npm run db:rotate-key`). GCM also detects tampering. Sign-in and duplicate checks use keyed hashes (HMAC-SHA256) of email and phone instead of plain text. Default rule (CLAUDE.md): **everything is encrypted unless the owner approves plain text**. |
| **Uploads** | Allowed types only, and the **file content must match its type** (signature check), size limits (images 10 MB, documents 50 MB, media 50 MB, max 6 per report), kept in memory and encrypted before storage. Random storage keys so file names never appear in paths. |
| **Rich text** | System update HTML is **sanitized on the server** (`sanitize-html`): only basic formatting tags; links limited to http, https and mailto; scripts, event handlers and every inline style except text alignment removed. |
| **Rate limiting** | 300 requests/min per IP; 400 requests/10 min per user; sign-in 3 failed attempts/10 min; forgot password 2/3 min; register & resend 5/hour; sign-in codes 10 wrong/10 min; uploads 6/10 min; support 5/10 min; invitations 20/hour. Over the limit → `429`. Sensitive limits are stored in Firestore (hashed keys) so restarts don't reset them. |
| **Errors & logging** | One error handler returns friendly messages; stack traces and internal errors are never sent to the browser. An activity log records sign-ins, sign-outs, password resets, role changes and every create/update/delete/download (no content). Users see their own log on the Profile page, and **10 deletions or 20 downloads in 10 minutes** email an alert. |
| **Duplicate accounts** | One account per email and per mobile number, checked inside a transaction at sign-up. |

## 4. Frontend: how it handles data

| Data | Where it is kept | How it is handled |
|---|---|---|
| Login token (JWT) | `localStorage` (`sca_token`) | Sent as `Authorization: Bearer` on every request by the axios instance. Signing out revokes it on the server, then removes it. Also removed when the server answers `401`, and after **30 minutes without activity**. |
| Profile and school details | Redux store (memory) | Loaded from `/auth/me` after sign-in; cleared on logout. Not written to browser storage. |
| Students, cases, appointments, files list | Component state (memory) | Fetched page by page (max 15) already decrypted by the server for the owner only; gone when the page closes. |
| Files, photos, media | Temporary blob URLs | Downloaded with the token (never a public link), shown via `URL.createObjectURL` and released after use. |
| Theme preference | `localStorage` | Light/dark setting only. No personal data. |
| Firebase config | Built into the app | Public by design (identifies the Firebase project for Google sign-in); it grants no database access. |
| Address lists | Memory cache | Public PSGC place lists (regions, cities, barangays). The chosen address is sent to our API, never to PSGC. |

## 5. Frontend security measures

- **No direct database access:** the browser only calls the API, so validation, encryption and access checks always run on the server.
- **Route guards:** signed-out users are sent to login; first-time users must complete their profile; developer pages are hidden unless the account has the `dev` role. (The backend checks roles too — the frontend only hides pages.)
- **Safe rendering:** React escapes all text by default. The only raw HTML shown is the system update content, which is sanitized on the server first.
- **Friendly errors only:** the axios layer turns failures into user-facing messages and never shows raw server errors.
- **Content Security Policy** (`vercel.json`): scripts only from the app, Google sign-in and analytics; connections only to the API, Firebase and PSGC; no plugins, no framing; plus `nosniff`, a strict referrer policy and a permissions policy.
- **Automatic sign-out** after 30 minutes of inactivity (all tabs together), and **Sign out of all devices** plus **Recent activity** on the Profile page.
- **Two-step sign-in** screen for the emailed code.
- **Client-side validation** (email, password length, PH mobile format, required address) gives quick feedback; the server validates again.
- **Google sign-in hardening:** `Cross-Origin-Opener-Policy: same-origin-allow-popups` header and a redirect fallback so the popup flow works safely on Vercel.
- **Upload checks before sending:** type and size limits are checked in the browser and again on the server.
- **Secrets:** only public `VITE_*` values are in the frontend; no API keys or encryption keys are shipped to the browser.

## 6. Infrastructure and secrets

| Secret | Where it lives | Purpose |
|---|---|---|

| | | |
|---|---|---|
| `ENCRYPTION_KEY` (+ `ENCRYPTION_KEY_PREVIOUS` during a rotation) | Render env / local `.env` | Encrypts and decrypts all protected data. Losing it = losing the data. |
| `JWT_SECRET` | Render env / local `.env` | Signs login tokens. Changing it signs everyone out. |
| `FIREBASE_SERVICE_ACCOUNT_JSON` / `sca.json` | Render env / local file (gitignored) | Full admin access to Firestore and Firebase Auth. |
| R2 access keys | Render env / local `.env` | Read/write the file bucket. |
| `BREVO_API_KEY` | Render env / local `.env` | Sends emails from the verified sender. |

`.env` and `sca.json` are in `.gitignore`. Dependency check (`npm audit`, production packages): **0 known issues** in both apps (frontend fixed with npm overrides for `@grpc/grpc-js`, `echarts` and `uuid`).

## 7. Possible security risks

FIXED handled in code OWNER needs an action only the owner can do ACCEPTED remaining, low impact.

| # | Risk | Status | What was done |
|---|---|---|---|
| **R1** | Firestore rules not deployed | OWNER | Deny-all rules are ready in `firestore.rules`; deploy them or paste them in Firebase console → Firestore → Rules. Until then, confirm the console shows `allow read, write: if false;`. |
| **R2** | Service-account key exposed | OWNER | Create a new key in Google Cloud, update Render and `sca.json`, delete the old key. |
| **R3** | One key decrypts everything | FIXED + OWNER | Key rotation added: `ENCRYPTION_KEY_PREVIOUS` + `npm run db:rotate-key` re-encrypt every value and file. Owner: keep an offline backup and give other Render members the Viewer role. |
| **R4** | Frontend dependency vulnerabilities | FIXED | Patched versions forced with npm overrides; `npm audit` reports 0. PowerPoint chart preview checked with ECharts 6. |
| **R5** | Token in localStorage | FIXED | Risk reduced: CSP blocks injected scripts, tokens can be revoked, idle sign-out after 30 minutes. (httpOnly cookies would need the frontend and API on the same domain, so they are not used.) |
| **R6** | No token revocation | FIXED | Session id + token version checked on every request: sign out, sign out of all devices and password reset end sessions immediately. |
| **R7** | No Content Security Policy | FIXED | CSP and security headers in `vercel.json`; tested with no violations on sign-in and registration. |
| **R8** | Plain-text fields remain | FIXED | User name, email, job title and school, support reports, system updates and invitation emails are encrypted; email lookups use a hash. Only system fields (ids, roles, statuses, times) stay plain. |
| **R9** | Upload type checked by extension only | FIXED | File signatures (magic bytes) checked for every upload. |
| **R10** | Any developer can grant the developer role | FIXED | Existing accounts must accept the emailed invitation; role changes appear in the activity log. |
| **R11** | No 2FA; simple passwords | FIXED | Emailed 6-digit code after the password; letters + numbers required; common passwords rejected. (Google sign-in uses Google's own 2-step verification.) |
| **R12** | Rich text allows style attributes | FIXED | Already limited to text alignment only (confirmed by test). |
| **R13** | Account enumeration at sign-up | FIXED | Sign-up always answers the same; the email owner is told by email. |
| **R14** | Rate limits in memory | FIXED | Sensitive limits stored in Firestore (hashed keys); only the two high-volume general limits stay in memory. |
| **R15** | Shared or unattended computers | FIXED | Automatic sign-out after 30 minutes without activity. |
| **R16** | Audit logs not reviewed | FIXED | Recent activity on the Profile page; email alert on bulk deletions or downloads. |
| **R17** | Email is the second factor and the reset channel | ACCEPTED | Someone with access to a user's mailbox can reset the password. Users should protect their email with 2-step verification. |

## 8. Recommended next steps

1. **Deploy:** push both apps. Everyone signs in once more (older tokens are rejected), now with the emailed code.
2. **Encrypt older records:** after the backend deploy, run `npm run db:encrypt-all` (users, support reports, updates, invitations).
3. **Owner actions:** deploy the Firestore rules (R1), rotate the service-account key (R2), back up `ENCRYPTION_KEY` offline and limit Render access (R3).
4. **Optional:** Firestore TTL policy on `rateLimits.expireAt`; a custom domain for frontend + API would allow httpOnly cookie sessions.